

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr.G.Ayyappan

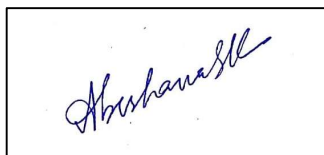
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5

7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

*I Abishana S K (192511242) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

		address the scenario			
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1) School Management System

Scenario Understanding

A School Management System stores information about students, classes, teachers, and grades. Each student belongs to one class, each class has one class teacher, and every student receives a grade. The objective is to retrieve the list of all students along with their respective class teacher and grade.

Identification of Relations (Relational Schema)

STUDENT

Attribute	Description
StudentID (PK)	Unique student ID
StudentName	Name of the student
ClassID (FK)	References CLASS table

CLASS

Attribute	Description
ClassID (PK)	Unique class ID
ClassName	Name of the class
TeacherID (FK)	References TEACHER table

TEACHER

Attribute	Description
TeacherID (PK)	Unique teacher ID
TeacherName	Name of the class teacher

GRADE

Attribute	Description
StudentID (PK, FK)	References STUDENT table
Grade	Student's grade

Relationship Between Tables

TEACHER
PK TeacherID
TeacherName

1:M

CLASS
PK ClassID
ClassName
FK TeacherID

1:M

STUDENT
PK StudentID
StudentName
FK ClassID

1:1

GRADE
PK/FK StudentID
Grade

SQL Query

```
SELECT
    S.StudentID,
    S.StudentName,
    C.ClassName,
    T.TeacherName AS ClassTeacher,
    G.Grade
FROM Student S
JOIN Class C
    ON S.ClassID = C.ClassID
JOIN Teacher T
    ON C.TeacherID = T.TeacherID
JOIN Grade G
    ON S.StudentID = G.StudentID;
```

Explanation

The query joins the Student, Class, Teacher, and Grade tables using their primary and foreign key relationships. It retrieves each student's ID, name, class, assigned class teacher, and grade. The use of INNER JOIN ensures that only students with valid class, teacher, and grade records are displayed.

Expected Output

StudentID	StudentName	ClassName	ClassTeacher	Grade
S101	Rahul	X-A	Mr. Kumar	A
S102	Priya	X-A	Mr. Kumar	B+
S103	Anjali	X-B	Ms. Meena	A+

Justification

The database is normalized by storing students, classes, teachers, and grades in separate relations, which minimizes data redundancy and improves data integrity. Foreign keys maintain referential integrity between the tables. Using JOIN operations efficiently combines related information to produce the required report.

2) Hospital Appointment Management System

Scenario Understanding

A hospital requires a database to manage patient appointments efficiently. Each patient can book appointments with doctors, and each doctor belongs to a specific department. The hospital wants to identify doctors who have handled more than 10 appointments during the current month for workload analysis and resource planning.

Relational Schema

PATIENT

Attribute	Description
PatientID (PK)	Unique patient ID
PatientName	Name of the patient
Age	Patient's age
Gender	Gender
Phone	Contact number

DOCTOR

Attribute	Description
DoctorID (PK)	Unique doctor ID
DoctorName	Doctor's name
Specialization	Medical specialization
DepartmentID (FK)	References DEPARTMENT table

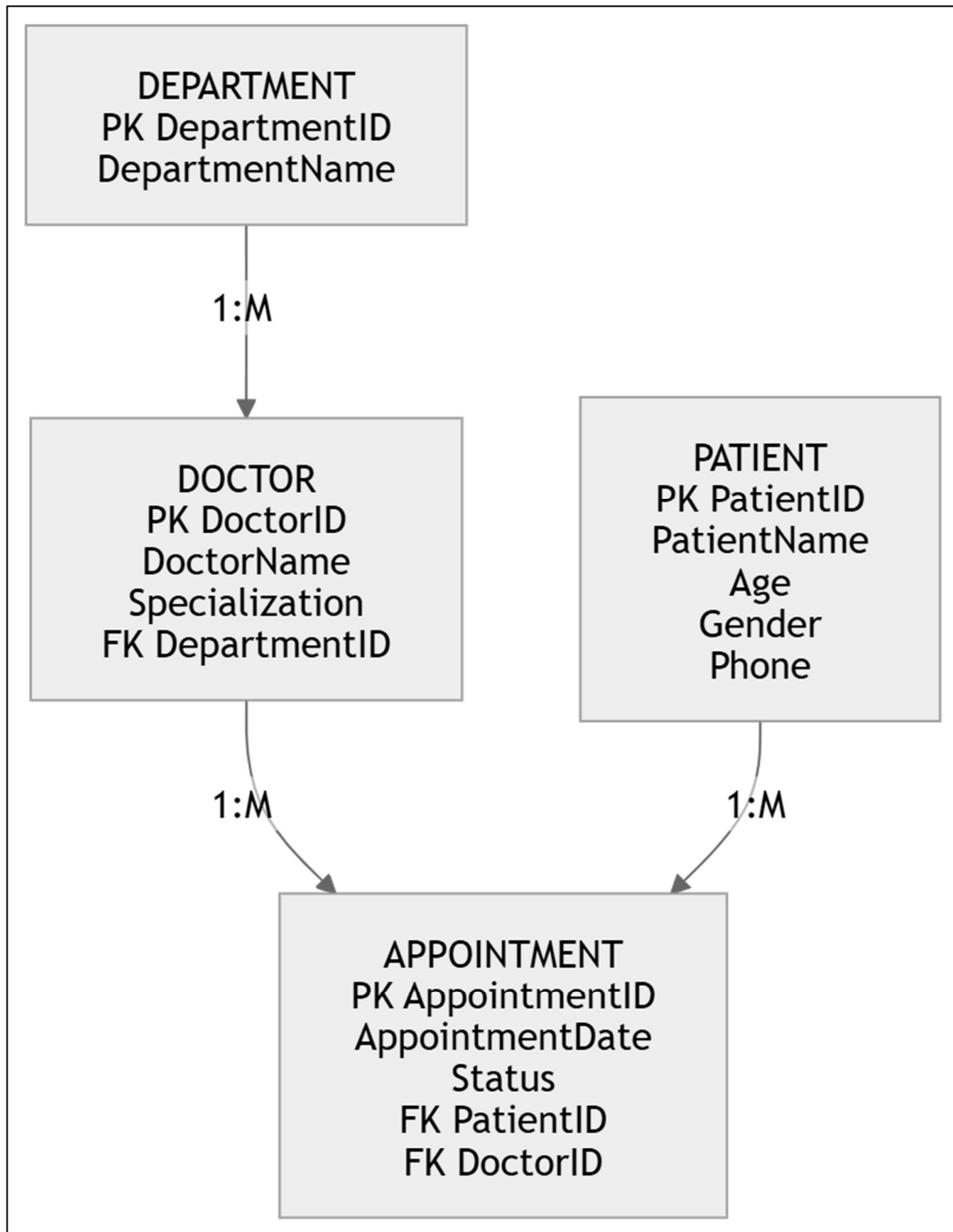
DEPARTMENT

Attribute	Description
DepartmentID (PK)	Unique department ID
DepartmentName	Name of department

APPOINTMENT

Attribute	Description
AppointmentID (PK)	Unique appointment ID
PatientID (FK)	References PATIENT table
DoctorID (FK)	References DOCTOR table
AppointmentDate	Date of appointment
Status	Scheduled/Completed/Cancelled

Relationship Between Tables



SQL Query

```
SELECT  
    D.DoctorID,  
    D.DoctorName,
```



```

COUNT(A.AppointmentID) AS TotalAppointments
FROM Doctor D
JOIN Appointment A
  ON D.DoctorID = A.DoctorID
WHERE MONTH(A.AppointmentDate) = MONTH(CURRENT_DATE)
  AND YEAR(A.AppointmentDate) = YEAR(CURRENT_DATE)
GROUP BY D.DoctorID, D.DoctorName
HAVING COUNT(A.AppointmentID) > 10;

```

Explanation

The query joins the Doctor and Appointment tables using the DoctorID foreign key. It filters appointments that occurred in the current month, groups the records by doctor, counts the number of appointments handled by each doctor, and returns only those doctors whose appointment count exceeds 10 using the HAVING clause.

Expected Output

DoctorID	DoctorName	TotalAppointments
D101	Dr. Meena	15
D104	Dr. Arjun	18
D107	Dr. Kavitha	12

Justification

The schema is normalized by storing patients, doctors, departments, and appointments in separate tables. Primary and foreign keys enforce referential integrity and eliminate data redundancy. The use of JOIN, GROUP BY, COUNT(), and HAVING enables efficient retrieval of doctors with a high appointment load, supporting hospital administration in monitoring workload and allocating medical resources effectively.

3)

E-Commerce System

Scenario Understanding

An e-commerce company maintains records of products and customer orders. Some products may exist in the inventory but have never been purchased. The objective is to identify all products that have never been ordered using a sub-query.

Relational Schema

PRODUCT

Attribute	Description
ProductID (PK)	Unique product ID
ProductName	Name of the product
Category	Product category
Price	Product price

CUSTOMER

Attribute	Description
CustomerID (PK)	Unique customer ID
CustomerName	Customer name
Email	Email address

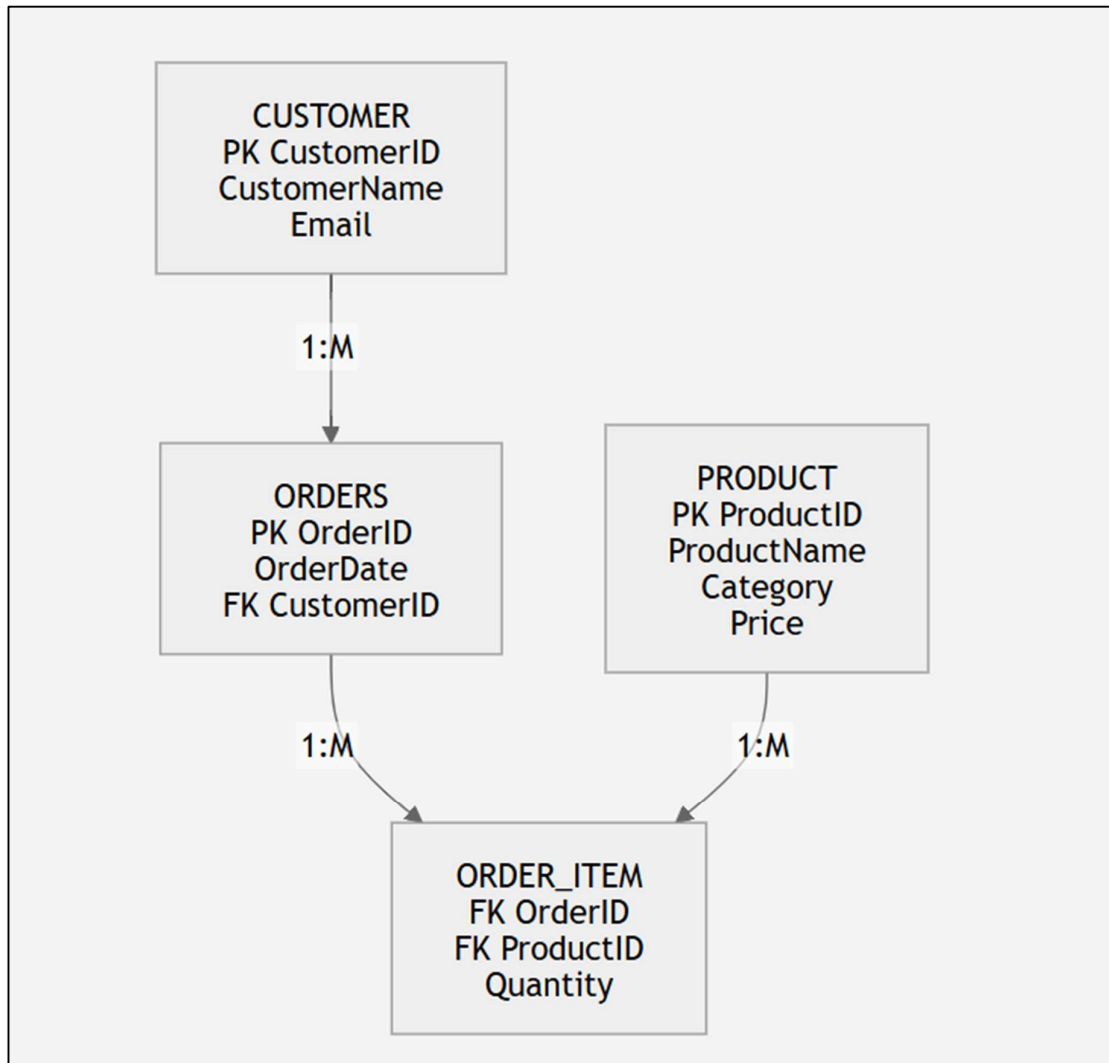
ORDERS

Attribute	Description
OrderID (PK)	Unique order ID
CustomerID (FK)	References CUSTOMER table
OrderDate	Date of order

ORDER_ITEM

Attribute	Description
OrderID (FK)	References ORDERS table
ProductID (FK)	References PRODUCT table
Quantity	Quantity ordered

Relationship Between Tables



SQL Query (Using Sub-query)

```
SELECT ProductID, ProductName, Category, Price
FROM Product
WHERE ProductID NOT IN (
    SELECT ProductID
    FROM Order_Item
);
```

Alternative Query (Using NOT EXISTS)

```
SELECT P.ProductID,
       P.ProductName,
       P.Category,
       P.Price
```

```

FROM Product P
WHERE NOT EXISTS (
    SELECT 1
    FROM Order_Item OI
    WHERE OI.ProductID = P.ProductID
);

```

Explanation

The query retrieves all products that do not appear in the ORDER_ITEM table. The sub-query returns the ProductIDs that have been ordered, while the outer query selects only those products whose ProductID is not present in that list. This identifies products that have never been purchased.

Expected Output

ProductID	ProductName	Category	Price
P105	Wireless Mouse	Electronics	650
P112	Office Chair	Furniture	4500
P118	Water Bottle	Accessories	350

Justification

The schema separates products, customers, orders, and order details into normalized tables, reducing redundancy and maintaining data integrity through primary and foreign keys. A sub-query is used to compare the Product table with the Order_Item table, efficiently identifying products that have never been ordered. This information helps the company analyze unsold inventory and improve stock management.

4) Scenario Understanding

A university maintains records of students and the courses in which they are enrolled. A student can enroll in multiple courses, and each course can have multiple students. The objective is to identify students who are enrolled in both 'DBMS' and 'OS' (Operating Systems) courses using Relational Algebra.

Relational Schema

STUDENT

Attribute	Description
StudentID (PK)	Unique student ID
StudentName	Name of the student
Department	Student's department

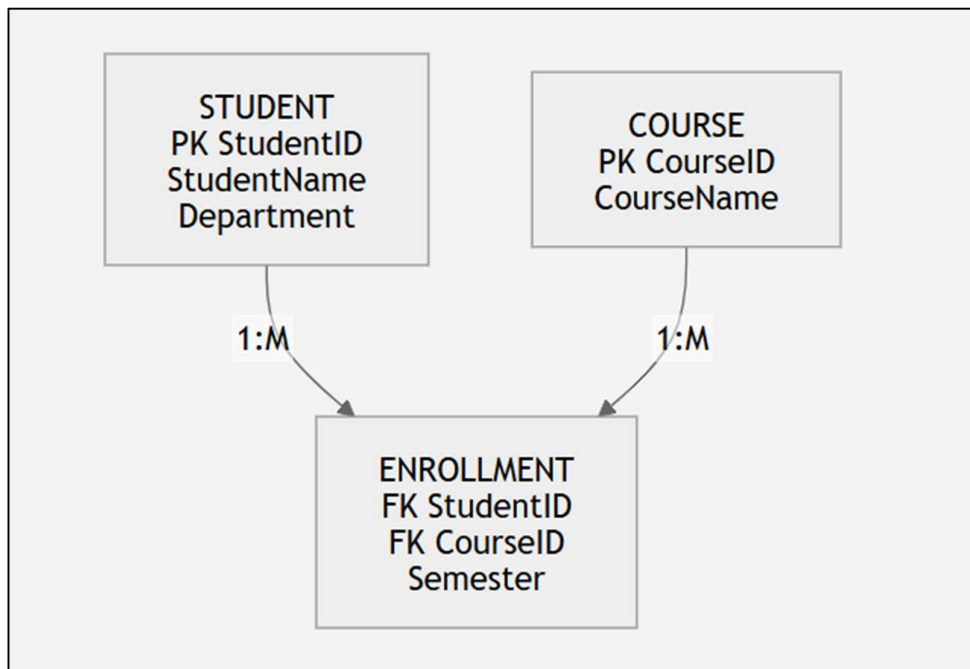
COURSE

Attribute	Description
CourseID (PK)	Unique course ID
CourseName	Name of the course

ENROLLMENT

Attribute	Description
StudentID (FK)	References STUDENT table
CourseID (FK)	References COURSE table
Semester	Semester of enrollment

Relationship Between Tables



Relational Algebra Expression

Step 1: Students enrolled in DBMS

DBMS_Students $\leftarrow \pi_{\text{StudentID}}$
($\sigma_{\text{CourseName}='DBMS'}$
(COURSE $\bowtie$ ENROLLMENT))

Step 2: Students enrolled in OS

OS_Students $\leftarrow \pi_{\text{StudentID}}$
($\sigma_{\text{CourseName}='OS'}$
(COURSE $\bowtie$ ENROLLMENT))

Step 3: Students enrolled in both DBMS and OS

Result $\leftarrow \text{DBMS_Students} \cap \text{OS_Students}$

Step 4: Display student details

$\pi_{\text{StudentID}, \text{StudentName}}$
(STUDENT $\bowtie$ Result)

Explanation

The COURSE table is joined with the ENROLLMENT table to determine which students are enrolled in the DBMS and OS courses. The selection (σ) operator filters records for each course, while the projection (π) operator extracts only the StudentID. The intersection ($\cap$) operator identifies students enrolled in both courses. Finally, the STUDENT table is joined with the result to retrieve the corresponding student details.

Expected Output

StudentID	StudentName
S101	Rahul
S108	Priya
S115	Arjun

Justification

The schema is normalized by storing students, courses, and enrollments in separate relations. The ENROLLMENT table resolves the many-to-many relationship between students and courses using foreign keys. Relational Algebra operators such as selection (σ), projection (π), join ($\bowtie$), and intersection ($\cap$) efficiently retrieve students who are enrolled in both the DBMS and OS courses while maintaining data integrity.

Payroll Management System

Scenario Understanding

A payroll management system stores employee salary details along with their department information. The organization wants to identify employees whose salary is below the average salary of their respective department. A view should be created to display such employees, and their salaries should then be increased by 10%.

Relational Schema

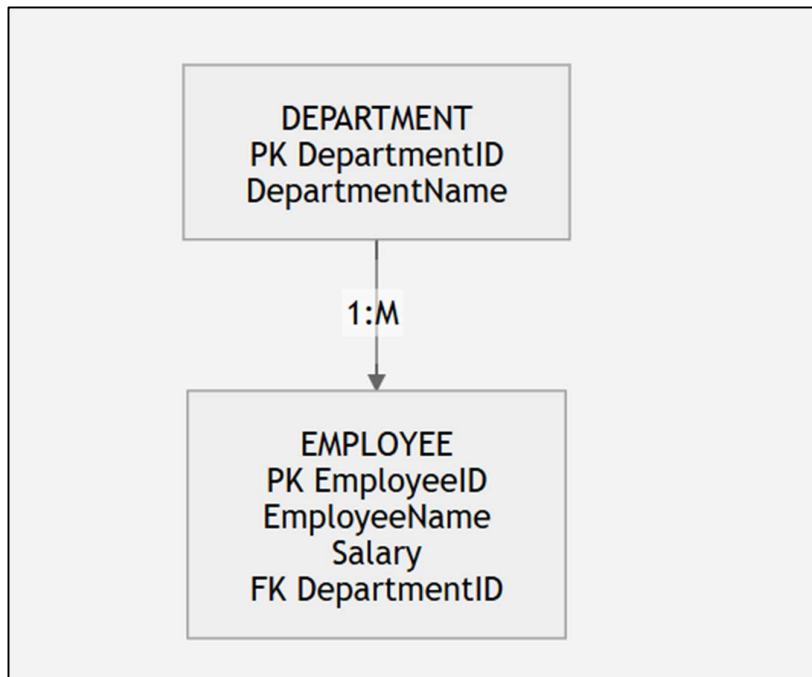
DEPARTMENT

Attribute	Description
DepartmentID (PK)	Unique department ID
DepartmentName	Name of the department

EMPLOYEE

Attribute	Description
EmployeeID (PK)	Unique employee ID
EmployeeName	Name of the employee
Salary	Employee salary
DepartmentID (FK)	References DEPARTMENT table

Relationship Between Tables



SQL Query

Step 1: Create View

```
CREATE VIEW BelowDeptAverage AS
SELECT
    E.EmployeeID,
    E.EmployeeName,
    E.Salary,
    E.DepartmentID
FROM Employee E
WHERE E.Salary <
(
    SELECT AVG(E2.Salary)
    FROM Employee E2
    WHERE E2.DepartmentID = E.DepartmentID
);
```

Step 2: Update Salary by 10%

```
UPDATE Employee
SET Salary = Salary * 1.10
WHERE EmployeeID IN
(
    SELECT EmployeeID
    FROM BelowDeptAverage
);
```

Explanation

The CREATE VIEW statement creates a virtual table named BelowDeptAverage, which contains employees whose salary is lower than the average salary of their respective department. The UPDATE statement then increases the salary of these employees by 10% by selecting their EmployeeIDs from the view.

Expected Output (View)

EmployeeID	EmployeeName	Salary	DepartmentID
E102	Priya	35000	D01
E108	Rahul	42000	D02

After Salary Update

EmployeeID	EmployeeName	Updated Salary
E102	Priya	38500
E108	Rahul	46200

Justification

The database is normalized by storing employee and department details in separate tables, reducing data redundancy. The view provides a reusable way to identify employees earning below their department's average salary without repeatedly writing the same query. The UPDATE statement applies a 10% salary increment only to employees identified by the view, ensuring efficient and accurate payroll processing.

6)

Banking Management System

Scenario Understanding

A banking system maintains records of customers, their bank accounts, branches, and transaction history. Each customer can own one or more accounts, each account belongs to a specific branch, and multiple transactions can be performed on an account. The objective is to retrieve customer account details, transaction history, and branch information using JOIN operations.

Relational Schema

CUSTOMER

Attribute	Description
CustomerID (PK)	Unique customer ID
CustomerName	Name of the customer
Phone	Contact number

BRANCH

Attribute	Description
BranchID (PK)	Unique branch ID
BranchName	Name of the branch
Location	Branch location

ACCOUNT

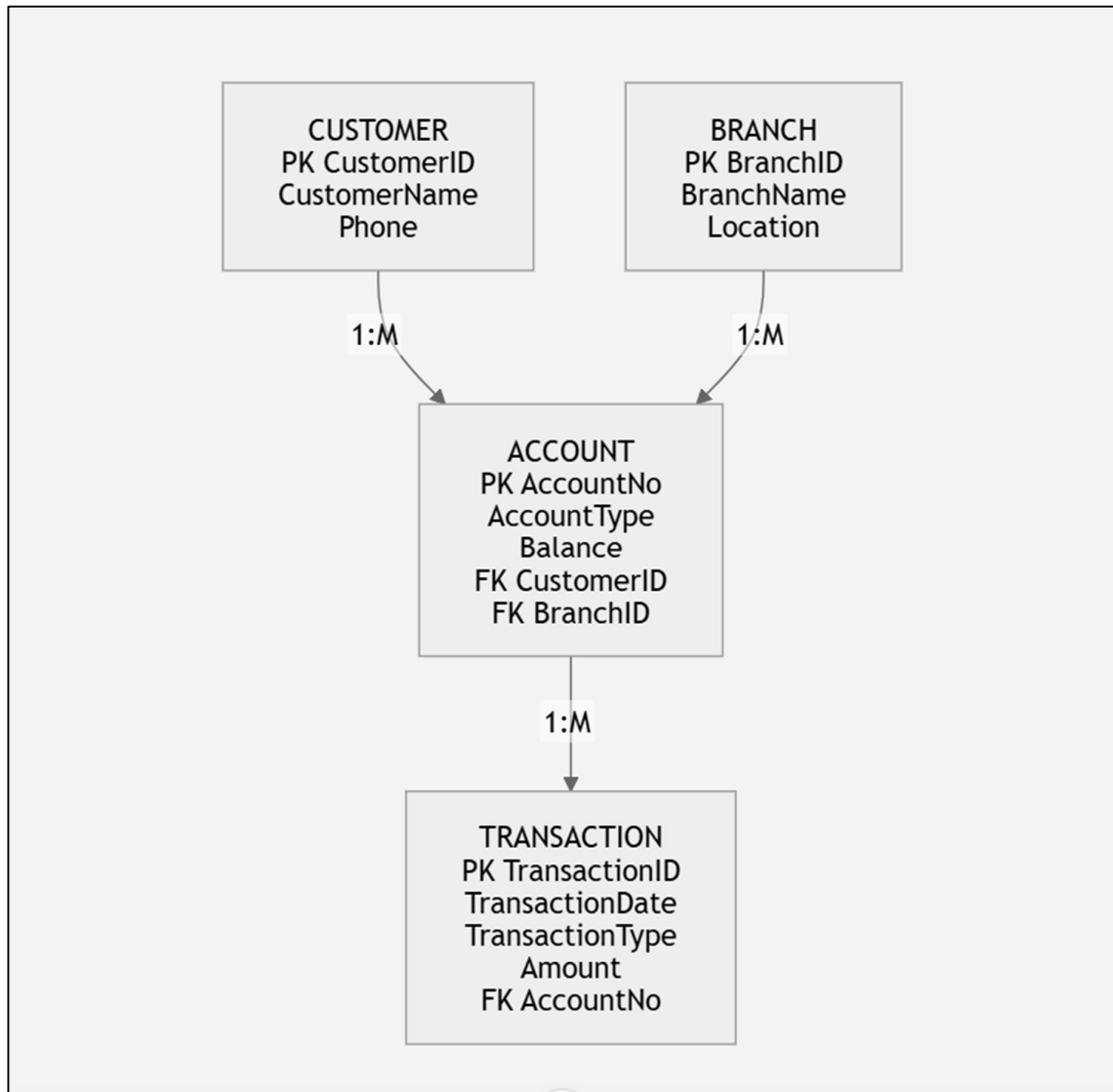
Attribute	Description
AccountNo (PK)	Unique account number
CustomerID (FK)	References CUSTOMER table

BranchID (FK)	References BRANCH table
AccountType	Savings / Current
Balance	Current account balance

TRANSACTION

Attribute	Description
TransactionID (PK)	Unique transaction ID
AccountNo (FK)	References ACCOUNT table
TransactionDate	Date of transaction
TransactionType	Credit / Debit
Amount	Transaction amount

Relationship Between Tables



SQL Query (Using JOIN Operations)

SELECT

C.CustomerID,
C.CustomerName,
A.AccountNo,
A.AccountType,
A.Balance,
T.TransactionID,
T.TransactionDate,
T.TransactionType,
T.Amount,
B.BranchName,

```

        B.Location
FROM Customer C
INNER JOIN Account A
    ON C.CustomerID = A.CustomerID
INNER JOIN Transaction T
    ON A.AccountNo = T.AccountNo
INNER JOIN Branch B
    ON A.BranchID = B.BranchID;

```

Explanation

The query uses INNER JOIN operations to combine data from the Customer, Account, Transaction, and Branch tables. The Customer table is joined with the Account table using CustomerID, the Account table is joined with the Transaction table using AccountNo, and the Account table is joined with the Branch table using BranchID. This retrieves complete customer account information along with transaction history and branch details.

Expected Output

Custo merID	Custome rName	Accou ntNo	Accoun tType	Bala nce	Transac tionID	Transacti onType	Amo unt	Branch Name	Loca tion
C101	Rahul	A1001	Savings	45,0 00	T201	Credit	5,00 0	Anna Nagar	Chen nai
C102	Priya	A1002	Current	72,0 00	T202	Debit	2,00 0	Tambar am	Chen nai
C101	Rahul	A1001	Savings	45,0 00	T203	Debit	1,50 0	Anna Nagar	Chen nai

Justification

The banking database is normalized by storing customers, accounts, branches, and transactions in separate tables. Primary and foreign keys maintain referential integrity and reduce data redundancy. The use of INNER JOIN operations efficiently combines related data from multiple tables, allowing the bank to generate a comprehensive report containing customer details, account information, transaction history, and branch information in a single query.

7) Retail Store Sales Management System

Scenario Understanding

A retail store records daily sales of products belonging to different categories. Management requires a daily sales summary to identify product categories whose total sales exceed Rs. 10,000. This helps in analyzing high-performing categories and making inventory and sales decisions.

Relational Schema

CATEGORY

Attribute	Description
CategoryID (PK)	Unique category ID
CategoryName	Name of the product category

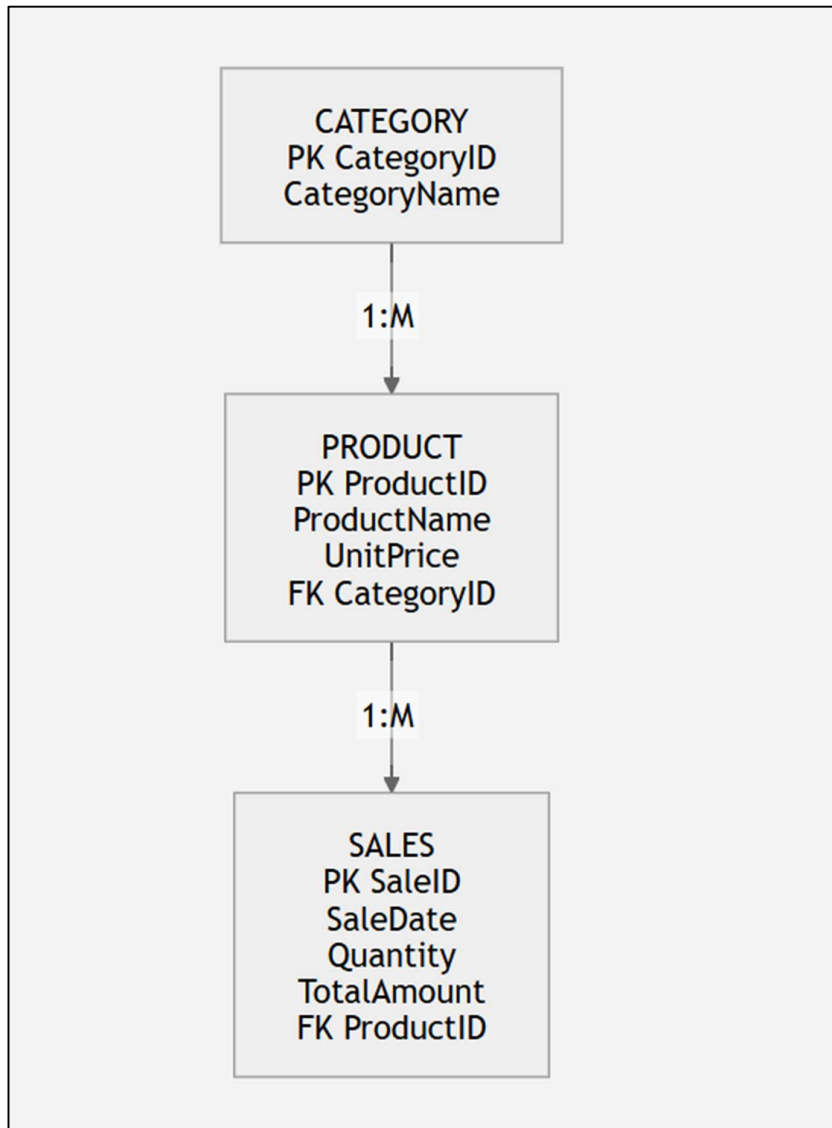
PRODUCT

Attribute	Description
ProductID (PK)	Unique product ID
ProductName	Name of the product
CategoryID (FK)	References CATEGORY table
UnitPrice	Price per unit

SALES

Attribute	Description
SaleID (PK)	Unique sale ID
ProductID (FK)	References PRODUCT table
SaleDate	Date of sale
Quantity	Quantity sold
TotalAmount	Total sale amount

Relationship Between Tables



SQL Query (Using GROUP BY and HAVING)

```
SELECT
    C.CategoryID,
    C.CategoryName,
    SUM(S.TotalAmount) AS TotalSales
FROM Category C
JOIN Product P
    ON C.CategoryID = P.CategoryID
JOIN Sales S
    ON P.ProductID = S.ProductID
WHERE S.SaleDate = CURRENT_DATE
GROUP BY C.CategoryID, C.CategoryName
```

HAVING SUM(S.TotalAmount) > 10000;

Explanation

The query joins the Category, Product, and Sales tables to retrieve sales information. The GROUP BY clause groups sales records by category, while the SUM() function calculates the total sales for each category. The HAVING clause filters the grouped results and displays only those categories whose total daily sales exceed Rs. 10,000.

Expected Output

CategoryID	CategoryName	TotalSales (Rs.)
C01	Electronics	25,500
C03	Groceries	14,800
C05	Clothing	18,200

Justification

The retail database is normalized by storing categories, products, and sales in separate tables, ensuring data consistency and reducing redundancy. The use of JOIN operations combines related information from multiple tables, while GROUP BY summarizes sales by category and HAVING filters only those categories whose total daily sales exceed Rs. 10,000. This enables the retail store to quickly identify high-performing product categories and supports effective inventory and business decision-making.

8) Library Management System

Scenario Understanding

A library maintains records of its members, books, and borrowing transactions. Each book belongs to a specific genre, and members can borrow books from different genres. The objective is to identify members who have borrowed books from all available genres using Tuple Relational Calculus (TRC).

Relational Schema

MEMBER

Attribute	Description
MemberID (PK)	Unique member ID

MemberName	Name of the member
Phone	Contact number

GENRE

Attribute	Description
GenreID (PK)	Unique genre ID
GenreName	Name of the genre

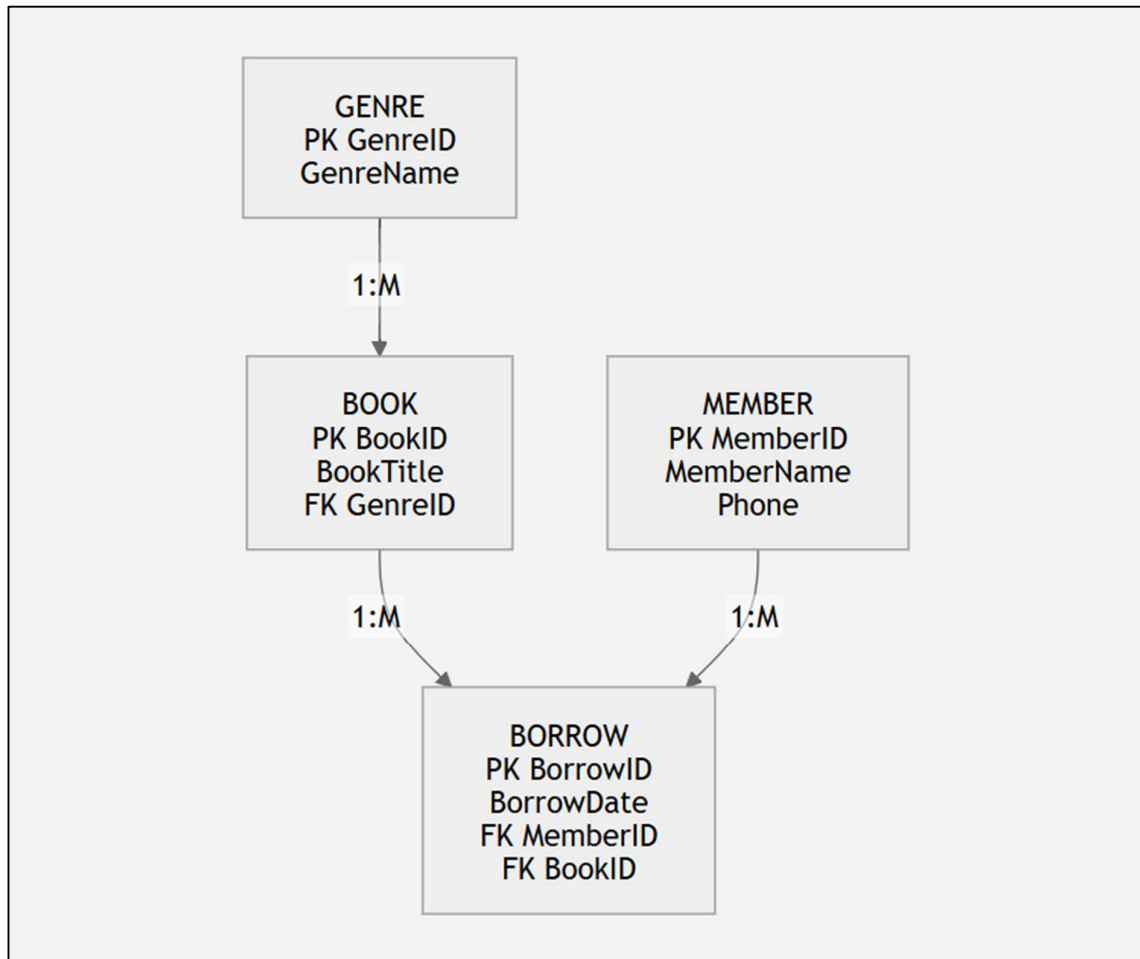
BOOK

Attribute	Description
BookID (PK)	Unique book ID
BookTitle	Title of the book
GenreID (FK)	References GENRE table

BORROW

Attribute	Description
BorrowID (PK)	Unique borrow ID
MemberID (FK)	References MEMBER table
BookID (FK)	References BOOK table
BorrowDate	Date of borrowing

Relationship Between Tables



Tuple Relational Calculus (TRC) Expression

```

{ M.MemberName |
  MEMBER(M) ∧
  ∀G (GENRE(G) →
    ∃B ∃R (
      BOOK(B) ∧
      BORROW(R) ∧
      B.GenreID = G.GenreID ∧
      R.BookID = B.BookID ∧
      R.MemberID = M.MemberID
    )
  )
}
  
```

Explanation

The TRC expression retrieves the names of members who have borrowed books from every genre available in the library. The universal quantifier ($\forall$) ensures that every genre is considered, while the existential quantifier ($\exists$) checks that the member has borrowed at least one book belonging to each genre.

Expected Output

MemberID	MemberName
M101	Rahul
M108	Priya
M115	Arjun

Justification

The library database is normalized by storing members, genres, books, and borrowing records in separate relations. Primary and foreign keys maintain referential integrity and eliminate data redundancy. The Tuple Relational Calculus expression uses universal ($\forall$) and existential ($\exists$) quantifiers to accurately identify members who have borrowed books from all genres, making it suitable for advanced database querying and analysis.

9)

Inventory Management System

Scenario Understanding

An inventory management system stores information about suppliers and products. Each product is supplied by a registered supplier. To ensure data integrity, the system enforces CHECK constraints to validate product details and FOREIGN KEY constraints to maintain valid relationships between products and suppliers.

Relational Schema

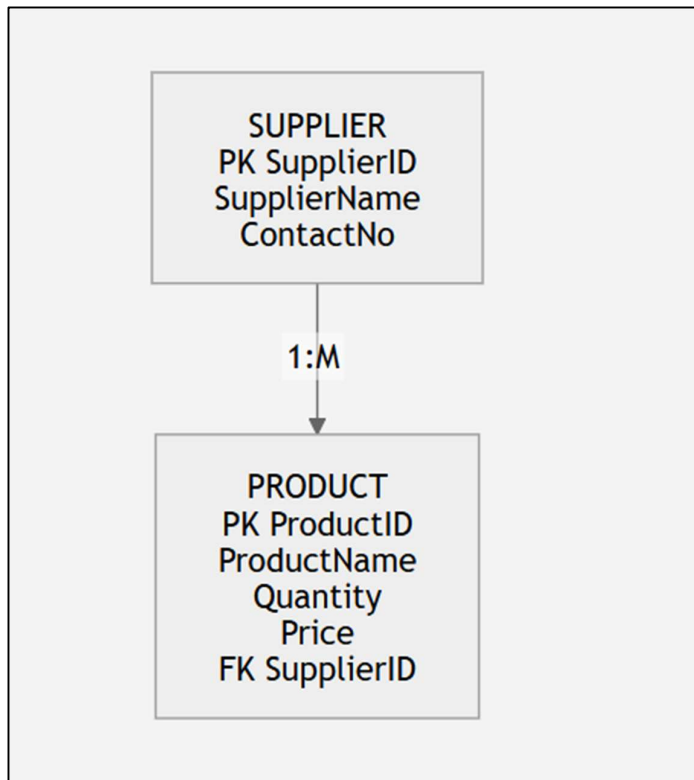
SUPPLIER

Attribute	Description
SupplierID (PK)	Unique supplier ID
SupplierName	Name of the supplier
ContactNo	Contact number

PRODUCT

Attribute	Description
ProductID (PK)	Unique product ID
ProductName	Name of the product
Quantity	Available stock
Price	Product price
SupplierID (FK)	References SUPPLIER table

Relationship Between Tables



SQL Statements

Step 1: Create Supplier Table

```

CREATE TABLE Supplier (
    SupplierID INT PRIMARY KEY,
    SupplierName VARCHAR(50),
    ContactNo VARCHAR(15)
);
  
```

Step 2: Create Product Table with Constraints

```

CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
  
```

```
    ProductName VARCHAR(50),  
    Quantity INT CHECK (Quantity >= 0),  
    Price DECIMAL(10,2) CHECK (Price > 0),  
    SupplierID INT,  
    FOREIGN KEY (SupplierID)  
        REFERENCES Supplier(SupplierID)  
);
```

Demonstrating Constraint Enforcement

Valid Insert (Success)

```
INSERT INTO Supplier  
VALUES (101, 'ABC Suppliers', '9876543210');
```

```
INSERT INTO Product  
VALUES (201, 'Keyboard', 50, 850.00, 101);
```

Result: Record inserted successfully.

CHECK Constraint Violation

```
INSERT INTO Product  
VALUES (202, 'Mouse', -10, 500.00, 101);
```

Result: Error because **Quantity cannot be negative** due to the CHECK (Quantity >= 0) constraint.

FOREIGN KEY Constraint Violation

```
INSERT INTO Product  
VALUES (203, 'Monitor', 15, 8500.00, 999);
```

Result: Error because **SupplierID 999 does not exist** in the Supplier table, violating the **FOREIGN KEY** constraint.

Explanation

The CHECK constraints ensure that product quantity cannot be negative and the product price must be greater than zero. The FOREIGN KEY constraint ensures that every product is associated with a valid supplier from the Supplier table. These constraints help maintain data accuracy and consistency within the inventory management system.

Expected Output

Supplier Table

SupplierID	SupplierName	ContactNo
101	ABC Suppliers	9876543210

Product Table

ProductID	ProductName	Quantity	Price	SupplierID
201	Keyboard	50	850.00	101

Justification

The inventory database is normalized by separating supplier and product information into different tables, reducing data redundancy and maintaining referential integrity. The **CHECK** constraints prevent invalid values such as negative quantities or zero prices, while the **FOREIGN KEY** constraint ensures that products are linked only to existing suppliers. Demonstrating both successful and failed insert operations verifies that these constraints are correctly enforced, improving the reliability and consistency of the database.

10) Human Resource (HR) Management System

Scenario Understanding

An HR Management System maintains records of employees, their departments, monthly attendance, and performance evaluations. To improve reporting efficiency, the organization requires materialized views that store monthly attendance summaries and employee performance summaries. These precomputed views enable faster report generation and decision-making.

Relational Schema

DEPARTMENT

Attribute	Description
DepartmentID (PK)	Unique department ID
DepartmentName	Name of the department

EMPLOYEE

Attribute	Description
EmployeeID (PK)	Unique employee ID
EmployeeName	Name of the employee
DepartmentID (FK)	References DEPARTMENT table

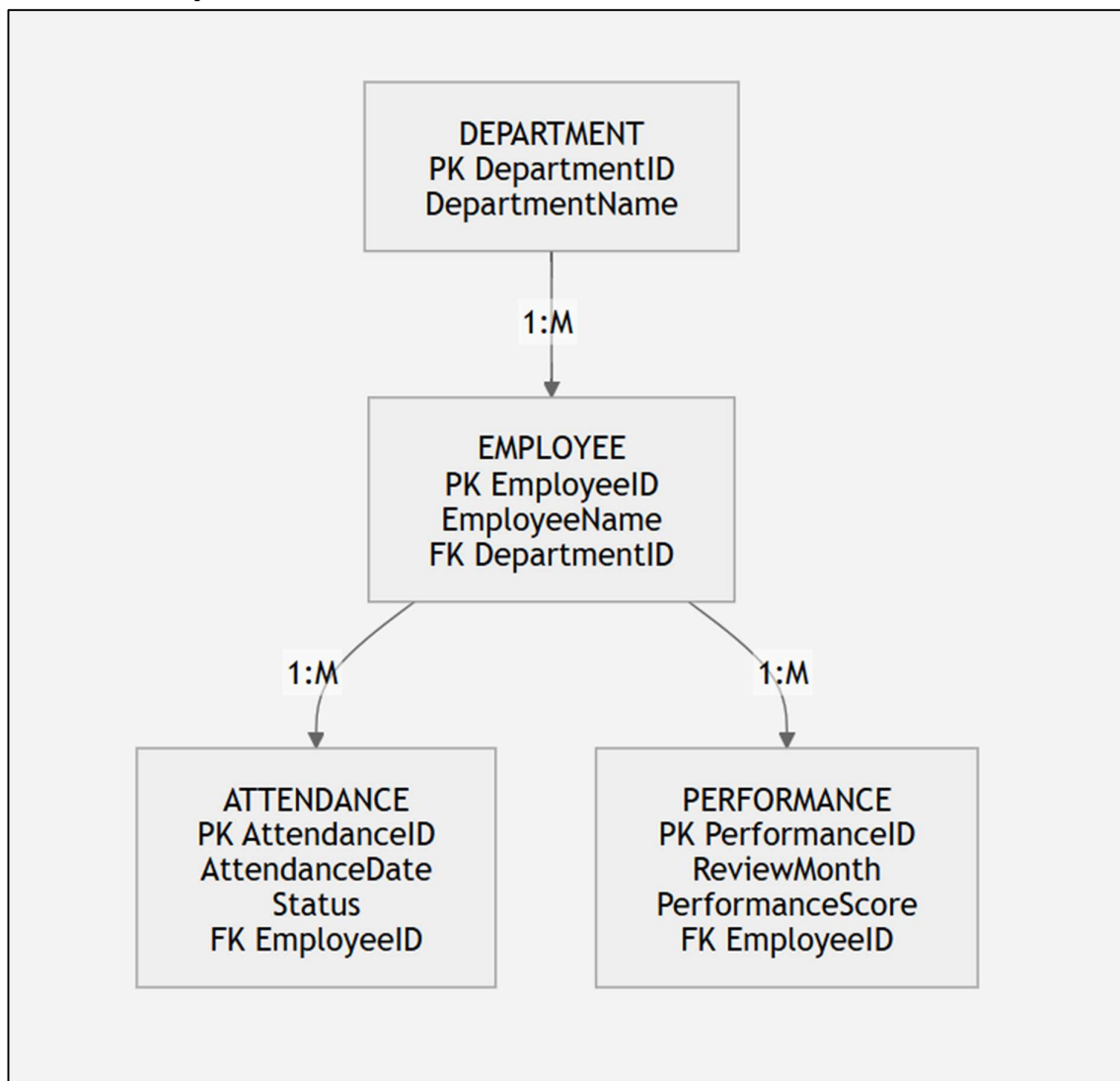
ATTENDANCE

Attribute	Description
AttendanceID (PK)	Unique attendance record
EmployeeID (FK)	References EMPLOYEE table
AttendanceDate	Date of attendance
Status	Present / Absent

PERFORMANCE

Attribute	Description
PerformanceID (PK)	Unique performance record
EmployeeID (FK)	References EMPLOYEE table
ReviewMonth	Evaluation month
PerformanceScore	Employee performance score

Relationship Between Tables



SQL Statements

Step 1: Materialized View for Monthly Attendance Summary

```
CREATE MATERIALIZED VIEW Monthly_Attendance_Summary
BUILD IMMEDIATE
REFRESH COMPLETE
AS
SELECT
    EmployeeID,
    TO_CHAR(AttendanceDate, 'YYYY-MM') AS Month,
    COUNT(*) AS TotalWorkingDays,
    SUM(CASE WHEN Status = 'Present' THEN 1 ELSE 0 END) AS
DaysPresent
FROM Attendance
GROUP BY EmployeeID,
    TO_CHAR(AttendanceDate, 'YYYY-MM');
```

Step 2: Materialized View for Monthly Performance Summary

```
CREATE MATERIALIZED VIEW Monthly_Performance_Summary
BUILD IMMEDIATE
REFRESH COMPLETE
AS
SELECT
    EmployeeID,
    ReviewMonth,
    AVG(PerformanceScore) AS AveragePerformance
FROM Performance
GROUP BY EmployeeID, ReviewMonth;
```

Explanation

The Monthly_Attendance_Summary materialized view stores the total working days and the number of days an employee was present for each month. The Monthly_Performance_Summary materialized view stores the average monthly performance score for every employee. Since the data is precomputed and stored physically, reports can be generated much faster than executing the aggregation queries repeatedly.

Expected Output

Monthly_Attendance_Summary

EmployeeID	Month	TotalWorkingDays	DaysPresent
E101	2026-08	26	24
E102	2026-08	26	25

Monthly_Performance_Summary

EmployeeID	ReviewMonth	AveragePerformance
E101	Aug-2026	8.8
E102	Aug-2026	9.2

Justification

The HR database is normalized by storing departments, employees, attendance records, and performance evaluations in separate tables, ensuring data consistency and minimizing redundancy. Materialized views physically store summarized attendance and performance data, significantly improving the speed of report generation. They are especially useful for HR systems where monthly reports are accessed frequently but the underlying data changes less often.